Implementing a Multi-Resolution Convolutional Neural Network to Predict Eye Fixations

Smaranda Maria Tasmoc
Unit Code COMSM0045, Student Number [redacted]
University of Bristol - Department of Computer Science
Bristol, United Kingdom
[redacted]

1. Introduction

Computer Vision scientists have repeatedly tried to mathematically model the human visual attention mechanism to predict human eye fixations. Liu et al. [2] introduced the concept of a multi-resolution convolutional neural network, which may be able to compute both top-down and bottom-up features at the same time. Their work was motivated by limitations identified in previous studies. According to "Predicting Eye Fixations using CNN," the authors found that a model incorporating both top-down and bottom-up information from an image is more effective in predicting the regions where the human eye is likely to fixate. Earlier approaches usually attempted to maximize either top-down information or bottom-up features independently, without integrating them.

Liu et al. also highlighted several issues with traditional methods. The reliance on hand-crafted features is time-consuming, and it presents challenges when working with large-scale datasets due to a lack of automation. Additionally, many past approaches depend heavily on expertise in the field, making them inadequate for handling extensive data efficiently.

2. Related Work

Recently, novel deep neural network architectures designed to predict fixation locations have emerged. Deep Fix [3] automatically learns features hierarchically, utilizing "Location Convolutional" filters with holes to capture global context through large receptive fields. It also extracts complex semantic features at multiple scales. Cornia et al. [4] proposed a new deep architecture for generating saliency predictions. Their model learns a non-linear combination of medium and high-level features extracted from a CNN. Wang et al. [5] introduced a visual attention network to predict human eye fixations using supervision, which is incorporated at multiple layers to achieve multi-level saliency predictions. This model is based on an encoder-decoder architecture in a fully convolutional manner, resulting in increased efficiency in both time and memory usage.

3. Dataset

To implement the work of Liu et al., I utilized the MIT dataset [1], applying 10-fold cross-validation. In this process, nine partitions were used for training, while one partition was kept for testing. The dataset, collected in 2009, contains an eye-tracking database that includes 1,003 images along with their corresponding ground truth fixation maps. These images depict a variety of subjects, such as natural scenes, indoor settings, everyday objects, textures, faces, landscapes, and abstract patterns. The primary objective of the dataset is to facilitate saliency map evaluation.

Accompanying the images is a dataset that includes fixation data, represented as 2D-coordinate data points indicating where the human eye fixated while viewing each image. In total, there are 2,100 gaze points distributed across 15 categories per image. This fixation data is used to generate the ground-truth saliency maps for each image, which are created by applying a Gaussian kernel to the fixation points. The ground truth fixation maps can be accessed on the source website.

4. Multiresolution CNN Architecture (Liu et al.)

The Mr-CNN model is a deep convolutional neural network (CNN) designed to take an image as input and output a predicted fixation map, also known as a saliency map. This fixation map aims to predict the visual regions in an image that will likely attract human attention. The training process is conducted end-to-end and is supervised, utilizing labelled data from the MIT dataset. The proposed model consists of three main components: preprocessing images in the input layer, implementing a multi-resolution CNN architecture that uses decision maps for supervised learning during training, and finally, generating a saliency map for each image during testing.

The input images are sourced from the MIT dataset, as detailed in Section 3. Each input is an RGB image of arbitrary size, which undergoes preprocessing before being fed into the network. Images are input into the model in two ways: for training and testing. During training, the image is cropped to various scales to produce dimensions of 150x150, 250x250, and 400x400 pixels. Each of these three images is further cropped to create samples of size 42x42 pixels. These three 42x42 samples are then fed into the multi-resolution CNN. During testing, the same rescaling process is applied to the test image, yielding three samples of 150x150, 250x250, and 400x400 pixels, from which 50x50 pixel locations are extracted. This generates 2,500 points per sample that represent the centres of regions to be cropped, which results in 2,500 mini-crops of size 42x42 pixels for each sample. Thus, three sets of 2,500 mini-crops per sample are input into the multi-resolution network.

The multi-resolution CNN accepts three streams of information, each with the same architecture. Each stream processes an image shaped 42x42x3 (width x height x channels). The structure of one stream, illustrated in Table 1, comprises three convolutional layers, each preceded by a max-pooling layer. The output of the final pooling layer is then flattened into a fully connected (FC) layer of 512 neurons. The FC layers from the three streams are combined into a single FC layer with 512 neurons. A sigmoid function is applied to this final FC layer, producing a binary label of 1 or 0, where 1 indicates a fixation point, and 0 represents a non-fixation point. The first convolutional layer employs 96 kernels of size 7x7, the second convolutional layer uses 160 kernels of size 3x3, and the third utilizes 288 kernels of size 3x3. The kernel stride is set to one, and the pooling layers use 2x2 windows, resulting in the following network architecture: I[42×42×3(×3)] - C[36×36×96(×3)] - P[18×18×96(×3)] - C[16×16×160(×3)] - P[8×8×160(×3)] - C[6×6×288(×3)] - P[3×3×288(×3)] - FC[512(×3)] - FC[512] - O[1].

As a supervised model, the Mr-CNN requires labelled data for both training and testing. During training, the labels are derived from a density map. During testing, the network outputs a down-sampled saliency map as a 50x50 matrix, where each of the 2,500 points in this down-sampled saliency map corresponds to the predictions of the mini-crops centred at the respective points. This down-sampled saliency map is then resized to match the dimensions of the original image, which is compared to the ground truth fixation map to calculate an AUC score.

5. Implementation Details

Platform and Routine:

- The proposed model was implemented using Python, CUDA 12.4.1, and Google Collab. The training utilized 8 CPUs and 2 GPUs, operating on an x86_64 architecture with a CPU speed of 3300 MHz

Data Processing:

- During training, 10 fixation locations and 20 non-fixation locations were sampled for each training image. This sampling was based on saliency values in the eye fixation density maps, with values greater than 0.9 classified as fixation locations and those less than 0.1 classified as non-fixation locations.
- When extracting image regions based on the center locations, if the center pixel was near the image borders, there might be insufficient pixels for extraction. In such cases, the borders of the image were copied to form regions of the correct size.

CNN Parameters and Settings:

- The model was trained and tested on the dataset using 10-fold cross-validation. The dataset was averaged and randomly divided into 10 partitions, with 9 partitions designated for training and 1 partition for testing.
- Additionally, 1/9 of the training set was reserved as a validation set to prevent overfitting.
- The performance of the Mr-CNN was evaluated every 2 training epochs, and the best-performing network, which exhibited minimal cross-entropy loss on the validation set, was selected.
- The mini-batch size was set to 256 for the MIT dataset.
- A momentum value of 0.9 was applied during training.
- To mitigate overfitting, dropout was implemented with a corruption probability of 0.5 in the third convolutional layer and the subsequent two fully connected layers across all networks.
- The network was trained using the MIT dataset, starting with a learning rate of 0.002, which decayed as training progressed.

6. Training Curves

Figure 1 and Figure 2 are the Tensor Board plots from the default version of the implementation, that do not have any of the additions from Section 5. The accuracy oscillates during training because the method for calculating the training loss is different than the method for calculating the validation loss. During training, the loss function performs cross-entropy on binary labels. The same cross-entropy loss function cannot be updated during validation to optimize the model, because the validation loop outputs a saliency map (a matrix of predictions from the model), which is compared against the ground truth fixation map. The remaining results were generated based on the information provided in Section 5.

The receiver operating characteristic (ROC) curve is a visual representation of a model's performance across all classification thresholds. The ROC curve is created by calculating the true positive rate (TPR) and the false positive rate (FPR) at every possible threshold (in practice, calculations are done at selected intervals), and then graphing TPR against FPR.

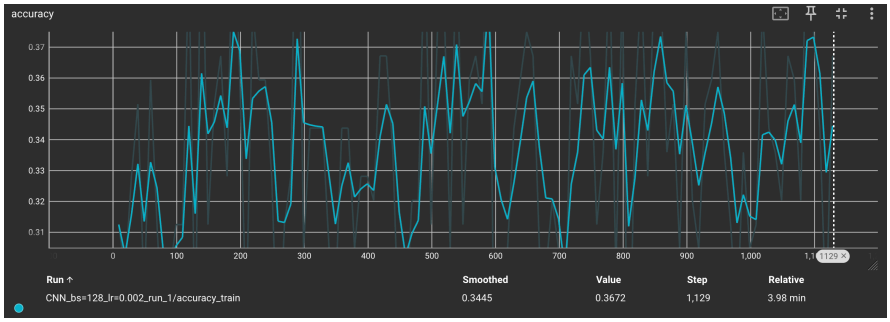

Figure 1 The training accuracy fluctuates between 31% and 37%, indicating that the weights are not being updated effectively.

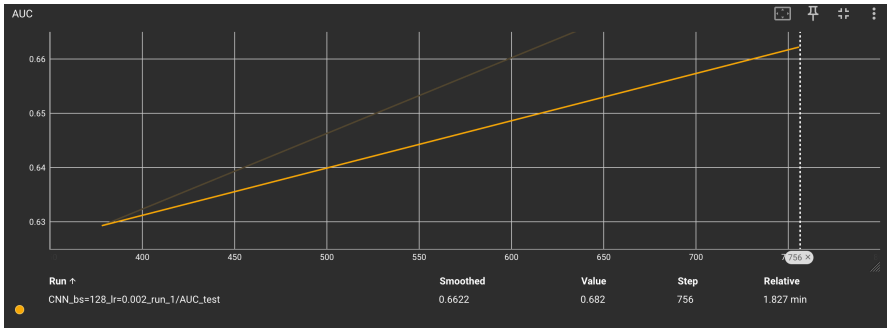

Figure 2 shows that the AUC score from the "default" program on the validation dataset increases linearly and reaches 68%.

The area under the curve (AUC) indicates the probability that the model will rank a randomly chosen positive example higher than a randomly chosen negative example. An AUC value of 1.0 signifies a 100% probability that the model will correctly rank a positive example above a negative one. For a binary classifier, if a model performs no better than random guessing, its ROC will appear as a diagonal line from (0,0) to (1,1), resulting in an AUC of 0.5, which corresponds to a 50% probability of correctly ranking a random positive and negative example.

In my case, the AUC score for the model is 91.62% (Fig. 3) based on multiple runs. However, the training accuracy (Fig. 4) fluctuates between 65% and 70%, indicating clear signs of underfitting. Additionally, the training loss (Fig. 5) does not converge toward 0%, suggesting that a learning rate of 0.002 is too low, causing the model to become trapped in a local minimum.

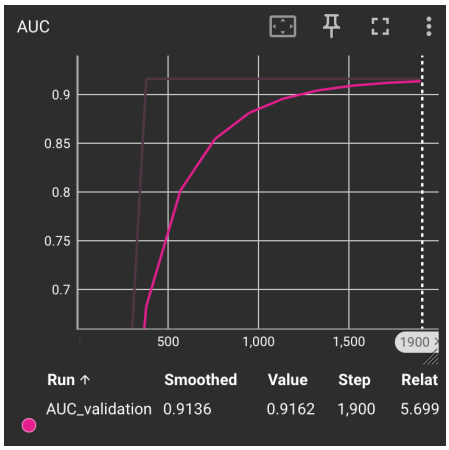

Figure 3 AUC achieved a score of 91.62% on validation, trained for 20 epochs, with a batch size of 256, employing learning rate decay, and an initial learning rate of 0.002.

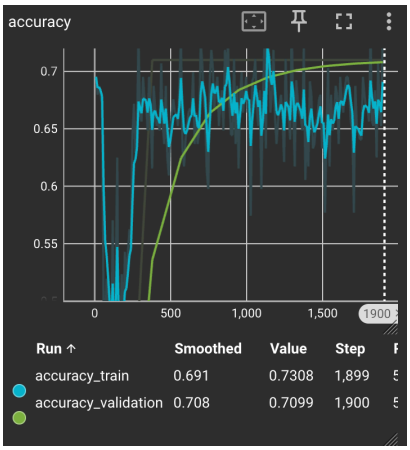

Figure 4 shows that the training accuracy exhibits erratic behavior, while the validation accuracy converges smoothly.

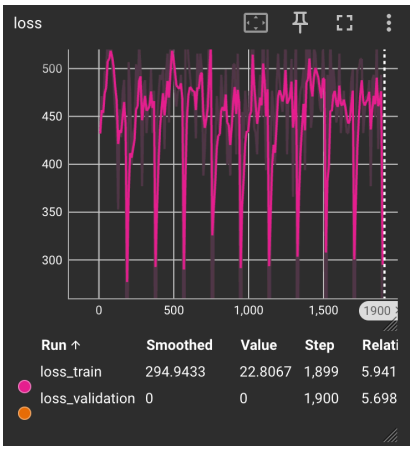

Figure 5 shows that the training loss is not informative because it does not exhibit the desired convergence pattern.

To further investigate, I examined the model's precision (Fig. 7) and sensitivity (Fig. 6). Both metrics plateau at 0%, indicating that the true positive rate drops to zero at some point before step 500. Interestingly, during the initial epochs, the sensitivity consistently remains at 100%, which means the false negative rate is zero—a positive outcome.

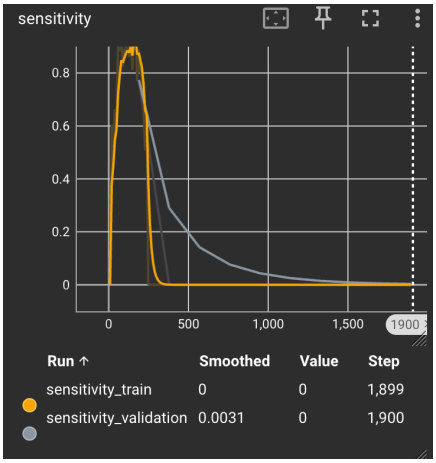

Figure 6 illustrates a decline in sensitivity during both the training and validation phases.

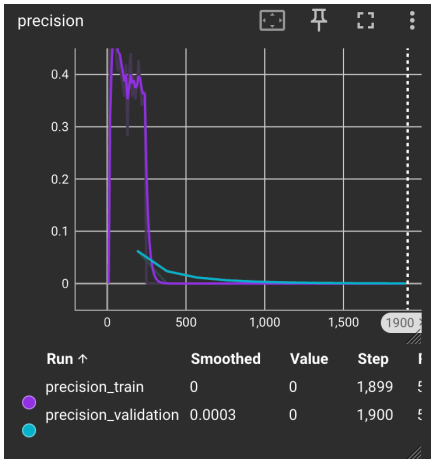

Figure 7 shows that during both training and testing, the precision drops to zero.

7. Conclusion and Future Work

To conclude, the work done by Liu et al. is an important advancement in the field of Computer Vision with the invention of a multi-resolution CNN that combines its top-down features with bottom-up features to predict eye fixations simultaneously. The next steps may require hyperparameter adjustments to help the model achieve the desired outcomes, which could include tuning learning rate, tuning batch size, batch normalization [7], and/or data augmentation. The results obtained in this report were not as expected, indicating the need for additional work to achieve more reliable outcomes.

8. References

[1] T. Judd, K. Ehinger, F. Durand, and A. Torralba, "Learning to predict where humans look," in *IEEE International Conference on Computer Vision (ICCV)*, 2009.

[2] Y. Liu, Z. Sun, and W. Lee, "Predicting eye fixations using convolutional neural networks," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015, pp. 362–370. https://doi.org/10.1109/CVPR.2015.7298604

[3] S. S. S. Kruthiventi, K. Ayush, and R. V. Babu, "DeepFix: A fully convolutional neural network for predicting human eye fixations," *IEEE Transactions on Image Processing*, vol. 26, no. 9, pp. 4446–4459, 2017. https://doi.org/10.1109/TIP.2017.2711331

[4] M. Cornia, L. Baraldi, G. Serra, and R. Cucchiara, "A deep multi-level network for saliency prediction," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 1–9. https://doi.org/10.1109/CVPR.2016.27

[5] W. Wang and J. Shen, "Deep visual attention prediction," *IEEE Transactions on Image Processing*, vol. 27, no. 3, pp. 1070–1082, 2018. https://doi.org/10.1109/TIP.2017.2774411

[6] G. E. Hinton, N. Srivastava, A. Krizhevsky, I. Sutskever, and R. R. Salakhutdinov. Improving neural networks by preventing co-adaptation of feature detectors. arXiv preprint arXiv:1207.0580, 2012.

[7] Ioffe, S., & Szegedy, C. (2015). Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift. *Proceedings of the 32nd International Conference on Machine Learning (ICML 2015)*, 37, 448-456.